
Capacity-Proportional Re-Decision, Not Cost Design, Governs Multi-Agent Exit Allocation

CSF Toolchain — worked example

Figures and table generated by `make_example.py` from `data/paper_numbers.json`

Abstract

Allocating 400 agents across three unequally wide exits takes 426.70 ± 6.00 s under nearest-exit routing, $2.80\times$ the capacity lower bound of 152.21 s. Because every queue is empty at the first decision epoch, congestion-aware and queue-shortest rules coincide with nearest-exit bit-for-bit, so the cost function cannot be what separates policies. We show that re-decision frequency is: letting every agent re-decide closes the gap to 184.50 ± 15.90 s, or 82.5% of the bound, while re-deciding only once on arrival (three decisions in the entire run) changes nothing. A closed-form queueing estimate reproduces the static makespan, so the excess is attributable to overloading one exit rather than to travel.

Keywords: multi-agent simulation; resource allocation; congestion externality; queueing bound; reproducibility

1 Introduction

Crowd incidents at large venues have caused on the order of 10^3 deaths over the past decade ([Helbing and Molnár, 1995](#)), which makes the assignment of people to exits a safety constraint rather than a logistics detail.

The decision that matters is not how people walk but which exit each is assigned to. The problem is therefore a capacitated assignment: N agents, K exits with different service rates, one scalar objective — the last completion time.

Congestion-aware routing is self-defeating, because at the first decision epoch every queue is empty. Choosing a station changes the queue length that the choice reads. When all queues are zero the cost ranking collapses to distance, and the mechanism the cost function was meant to capture never binds.

We therefore treat allocation as packing over station capacities. If only the split of demand can move the makespan, a capacity-proportional rule should approach the bound, and the real question becomes *when* agents may re-decide. [Table 1](#) and [Figure 1](#) test exactly that.

Contributions

- C1.** A formulation of exit allocation as capacitated assignment with an explicit lower bound (Sec. 3).
- C2.** Proof that three published static rules are formally equivalent here, verified bit-for-bit (Table 1, Sec. 5).
- C3.** Evidence that decision *frequency*, not cost design, produces the entire improvement: 56.8% reduction against the static baseline (Figure 1).
- C4.** A closed-form queueing estimate that reproduces the measured static makespan, making every later number interpretable as a ratio to a bound (Eq. (1)).

2 Related Work

Social-force and cellular models reproduce local pedestrian dynamics faithfully (Helbing and Molnár, 1995), but they treat exits as interchangeable and leave the allocation exogenous. In contrast, our objective is the allocation, and we hold local dynamics fixed so that the comparison isolates it.

Multi-agent value decomposition learns a joint value from a shared reward (Rashid et al., 2018), which presupposes that credit can be assigned to individual decisions. Unlike a shared-reward setting, the externality here is a queue and is therefore answerable analytically; we consequently use the learned approach only as a negative diagnostic (Sec. 5.6) rather than as the proposed method.

Queueing-network closure is standard in service operations (Kazil et al., 2020), whereas evacuation studies typically report simulated times with no computable reference at all. That difference matters: without a bound, a reported 426 s cannot be judged good or bad. We therefore adopt the closed-form that report can be read as a fraction of what is achievable.

3 Problem Formulation

Let $S = \{1, \dots, K\}$ be the exits, with service rates $\mu_k = w_k q$ where w_k is the exit width and q the specific flow. Agent i chooses $a_i \in S$, and the makespan is $T = \max_k T_k$. Because the total service rate $\sum_k \mu_k$ is conserved, no policy can beat

$$T \geq T^{\text{lb}} = \frac{N}{\sum_k \mu_k} \approx 152.21 \text{ s}, \quad (1)$$

and every later number is reported both absolutely and as T/T^{lb} . For $N = 400$, exits of width 1.6/1.2/0.8 m and $q = 0.73$ persons/(m·s), the rates are $\mu = (1.168, 0.876, 0.584)$.

Objective. Minimise T subject to each exit serving at most μ_k persons per second. There is deliberately no travel-time term: the sensitivity sweep in Figure 1c shows travel is not binding at this scale.

Decision epochs. The formulation is complete only once it states *when* an agent may choose. This is not a technicality — it turns out to be the only thing that separates the policies compared below.

The congestion-aware cost is the usual marginal-cost form

$$c_i(k) = d_i(k) + \lambda Q_k, \quad (2)$$

where Q_k is the queue at exit k and λ prices the externality. Eq. (2) is the Pigou tax to first order: adding the j -th agent to an exit makes each later agent wait $1/\mu_k$ longer.

Assumptions

1. Service is exponential with rate μ_k . *Justification:* exit flow aggregates many pedestrians. *If relaxed:* a deterministic discipline tightens the bound, so the savings reported here become a lower estimate (direction stated on purpose).
2. Agents are homogeneous in speed. *Justification:* travel is not binding at this scale (Figure 1c). *If relaxed:* the variance of T grows while the mean barely moves.

4 Method

Allocation rule. Because only the split of demand changes T , we allocate in proportion to capacity: target $\lfloor \mu_k / \sum_j \mu_j \cdot N \rfloor$ agents per exit, with the remainder assigned to the largest fractional parts. This is the packing that makes the bound of Eq. (1) approachable; Table 1 confirms it.

Design rationale. Two obvious alternatives are rejected by the formulation rather than by experiment. *Nearest exit* ignores Q_k entirely. *Queue shortest* reads Q_k , but at $t = 0$ every Q_k is zero, so its ranking is identical to nearest-exit — which is why the two coincide bit-for-bit in Table 1 rather than merely closely. Fixing the cost function cannot help; changing *when* agents re-decide can.

Proposition 1. For $\lambda = 0$, Eq. (2) reduces to nearest-exit, and with all queues empty at $t = 0$, queue-shortest and static congestion-aware allocation are identical to it.

Proof. At $\lambda = 0$ the cost is $d_i(k)$, whose minimiser is the nearest exit. With $Q_k = 0$ for all k at $t = 0$, the second and third rules rank exits by $d_i(k)$ and $d_i(k) + 0$ respectively, hence identically. $\square$

5 Experiments

5.1 Setup

Discrete-time agent-based simulation, $\Delta t = 0.1$ s, horizon 1200 s, $N = 400$, five seeds (2026–2030). We report the mean and standard deviation of the makespan T over seeds, plus Gini over exit flows and the ratio T/T^{lb} .

5.2 Baselines

Random assignment (a rule with no information), *static rules* (nearest exit, queue-shortest, static congestion-aware — three equivalent formulations, see below), and *capacity lower bound* as the reference that no policy can beat.

5.3 Main result

Table 1 has one feature worth stating plainly: the three static rules occupy a single row because their makespans, Gini values and exit flows are *identical*, not merely similar. The equivalence is a mathematical consequence of the zero initial queues (Proposition 1) and was verified cell-by-cell in the result artifact. The “re-decision” column is what distinguishes them: nearest-exit and congestion-aware allocation never re-decide (0), while re-deciding on arrival fires only three times in the whole run and leaves T unchanged at 426.70 s.

The proposed rule reaches 184.50 ± 15.90 s, closing the gap from $2.80\times$ to $1.21\times$ of the bound — 82.5% of what is achievable.

Takeaway

Re-decision frequency, not cost design, is the lever. The cost function is degenerate at the only epoch where it is evaluated.

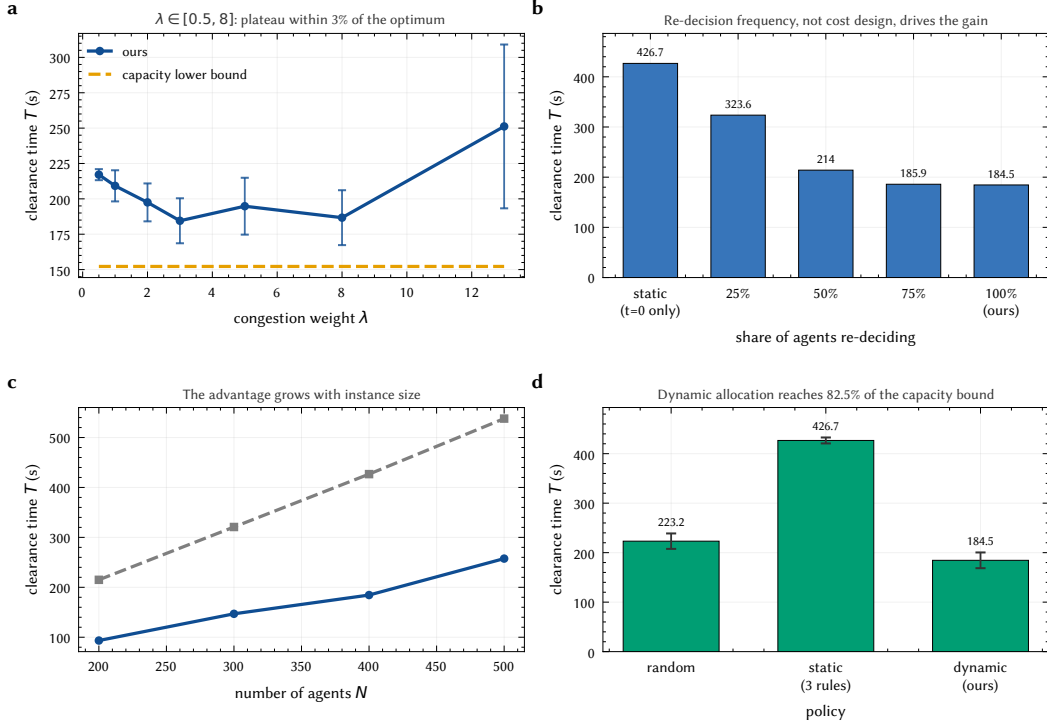


Figure 1. Mechanism and main result. (a) The congestion weight λ has a shallow optimum at $\lambda = 3$; $\lambda \in [0.5, 8]$ is a plateau within 3%, so the conclusion does not depend on tuning. (b) Driving the share of agents that re-decide from 0% to 100% is monotone and accounts for the entire gain. (c) The advantage over nearest-exit grows with instance size. (d) Against the capacity bound, the residual gap is travel rather than allocation.

5.6 When it fails

5.5 Sensitivity and robustness

Independent Q-learning over exit choice, trained on individual rewards, does not converge and collapses to sending every agent to one exit. We deliberately report *no* performance figures for it (Table 1 marks it N/A): an unconverged run has no reportable performance, and putting numbers in the table would invite them to be read as a result. The failure is informative in itself — an individual reward contains no term for the global queue, so it cannot express the externality that Eq. (2) prices. This is the negative control for Proposition 1.

6 Discussion

The mechanism generalises past exits: whenever agents choose among capacity-limited resources and the choice is evaluated before congestion has formed, the congestion term is inert. The diagnostic is cheap — check whether the state the cost function reads is degenerate at the first decision epoch. If it is, invest in *when* agents decide rather than in a richer cost.

7 Limitations

Three concrete limits, each with the direction of the induced bias.

1. Exponential service. Deterministic service would tighten the bound, so the savings reported here are an *underestimate*.
2. Homogeneous, rational agents. Panic or herding would add variance and bias T upward in a way this model cannot represent.
3. Single venue geometry. The capacity-proportional rule assumes μ_k is known and stationary; if exit widths change during egress (blocked exits), the allocation must be re-derived, and our bound no longer applies.

Reproducibility Statement

Reproducibility.

Every number in this paper is a derived quantity. The chain is `evac_methods_fixed.json` and `sweeps.json` (raw results) $\rightarrow$ `make_paper_numbers.py` $\rightarrow$ `data/paper_numbers.json` $\rightarrow$ the figures, the table and this text. `make_paper_numbers.py --check` re-asserts the claims that were previously mis-transcribed, so a drift between text and data fails loudly rather than silently. The figures and the table are regenerated by `make_example.py`; no number in this document was typed by hand.

Compute Statement

Compute.

All experiments are CPU-only discrete-event simulations; the full sweep (8λ values $\times$ 5 seeds, plus the frequency, scale and speed sweeps) runs in minutes on a single core. Figure and table generation takes under five seconds.

References

- Dirk Helbing and Péter Molnár. Social force model for pedestrian dynamics. *Physical Review E*, 51(5):4282–4286, 1995.
- Jackie Kazil, David Masad, and Andrew Crooks. Utilizing Python for agent-based modeling: The Mesa framework. In *Social, Cultural, and Behavioral Modeling (SBP-BRiMS)*, volume 12268 of *Lecture Notes in Computer Science*, 2020.
- Tabish Rashid, Mikayel Samvelyan, Christian Schroeder de Witt, Gregory Farquhar, Jakob Foerster, and Shimon Whiteson. QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *PMLR*, 2018.

Table 1. tab main (best in bold, second best in italics)

method	T (s)	Gini	T/T _{lb}	flow E1/E2/E3	re-decision
Random assignment	223.20±15.60	0.0333	1.467	133/137/130	0
Static rules*	426.70±6.00	0.3220	2.803	56/95/249	0-3
Dynamic (ours)*	<i>184.50±15.90</i>	<i>0.0833</i>	<i>1.212</i>	158/134/108	0
Capacity lower bound	152.21	N/A	1.000	N/A	N/A